プログラミング及び実習2

第 8 回試験（exam08）の解答例

佐野 彰

exam08 解答例: fill_grade.c（空欄を埋める・成績の段階）

```
    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        // ここから
        if (x >= 90) {
            count[0]++;
        } else if (x >= 80) {
            count[1]++;
        } else if (x >= 70) {
            count[2]++;
        } else if (x >= 60) {
            count[3]++;
        } else {
            count[4]++;
        }
        // ここまで
    }
```

- 上の段階から順に調べる。`else if` なので、たとえば `x >= 80` に来た時点で 90 未満であることは決まっている
- `count[0]` 〜 `count[4]` が S・A・B・C・F に対応する。境界の 90・80・70・60 は上の段階に入る

exam08 解答例: comment_sort.c（コメントで説明する・選択ソート）

```
    for (int i = 0; i < N - 1; i++) {
        // a[i] 以降の最小値の位置を、まず i と仮定する
        int min = i;
        // i より後ろの要素を順に調べて、最小値の位置を探す
        for (int j = i + 1; j < N; j++) {
            // それまでの最小値より小さい値が見つかったら、最小値の位置を j に更新する
            if (a[j] < a[min]) {
                min = j;
            }
        }
        // 見つけた最小値 a[min] と a[i] を入れ替える（tmp は入れ替えのための一時的な置き場所）
        int tmp = a[i];
        a[i] = a[min];
        a[min] = tmp;
    }
    // 小さい順に並んだ配列を出力する
```

- 「`j` を 1 ずつ増やす」のようにコードを言い換えるのではなく、**選択ソートの手順として何をしているか**を書く
- `min` は最小値そのものではなく、**最小値の位置**（添字）であることに注意する

exam08 解答例: runlength.c（ランレングス圧縮）

```
    int count = 0;
    for (int i = 0; i < n; i++) {
        count++;
        // 最後の値か、次の値が違うなら、ここまでの続きを出力する
        if (i == n - 1 || a[i + 1] != a[i]) {
            printf("%d %d\n", a[i], count);
            count = 0;
        }
    }
```

- 各要素をすぐ後ろの要素と比べ、違えば（または最後の要素なら）、そこで 1 つの組が終わる
- `i == n - 1` を**先に**調べる。`||` は左が真なら右を調べないので、`a[n]`（配列の外）を読まずにすむ
- 出力したら `count` を 0 に戻して、次の組を数え始める

exam08 解答例: medianmode.c（中央値と最頻値）

```c
#include <stdio.h>
#define N 100
#define MAX_SCORE 100

int main() {
    int n;
    int a[N];
    int count[MAX_SCORE + 1] = {0};
    scanf("%d", &n);
    if (n < 1 || n > N) {
        return 0;
    }
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        count[a[i]]++;
    }
    // 選択ソートで小さい順に並べる
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[min]) min = j;
        int t = a[i]; a[i] = a[min]; a[min] = t;
    }
    double median;
    if (n % 2 == 1) {
        median = a[n / 2];
    } else {
        median = (a[n / 2 - 1] + a[n / 2]) / 2.0;
    }

    int mode = 0;
    for (int v = 1; v <= MAX_SCORE; v++) {
        if (count[v] > count[mode]) {
            mode = v;
        }
    }
    printf("median %.1f\n", median);
    printf("mode %d %d\n", mode, count[mode]);
    return 0;
}
```

- 中央値は並べ替えてから真ん中を取る。偶数個なら中央の 2 つを `2.0` で割る（`2` だと int の割り算）
- 最頻値は点数ごとの個数 `count[点数]` で数える。`>` で比べるので、同じ個数なら小さい点数が残る

exam08 解答例: fix_dice.c（誤りを直す・さいころの目の数え上げ）

```
    if (n < 1 || n > N) {  // 修正
        return 0;
    }
    for (i = 0; i < n; i++) {  // 修正
        scanf("%d", &x);
        count[x]++;
    }
    for (k = 1; k <= 6; k++) {  // 修正
        printf("%d: %d\n", k, count[k]);
    }
    int mode = 1;
    for (k = 2; k <= 6; k++) {
        if (count[k] > count[mode]) {  // 修正
            mode = k;
        }
    }
```

1. `n < 1 && n > N` は決して真にならない → `||`
2. `i <= n` だと n + 1 個読もうとする → `i < n`
3. `k < 6` だと 6 の目を出力しない → `k <= 6`
4. `>=` だと、同じ回数のとき大きい目に置き換わる → `>`（最も小さい目を残す）

exam08 解答例: second.c（配列を使わずに 2 番目に大きい値）

```
    scanf("%d", &x);
    int max = x, second = x;  // 2 番目がまだ決まっていない間は second == max
    for (int i = 1; i < n; i++) {
        scanf("%d", &x);
        if (x > max) {
            second = max;
            max = x;
        } else if (x < max && (second == max || x > second)) {
            second = x;
        }
    }
    printf("max %d\n", max);
    if (second == max) {
        printf("second none\n");
    } else {
        printf("second %d\n", second);
    }
```

- 最大値が更新されるときは、それまでの最大値が 2 番目に下がる
- `x == max` のときは何もしない（同じ値は 1 つと数える）。2 番目が決まっていない印として `second == max` を使う

exam08 解答例: charcount.c（文字の種類の数え上げ）

```
    char c;
    int upper = 0, lower = 0, digit = 0, other = 0;
    while (1) {
        scanf("%c", &c);
        if (c == '#') {
            break;
        }
        if (c >= 'A' && c <= 'Z') {
            upper++;
        } else if (c >= 'a' && c <= 'z') {
            lower++;
        } else if (c >= '0' && c <= '9') {
            digit++;
        } else {
            other++;
        }
    }
```

- `%c` は空白も 1 文字として読む（`%d` と違って読み飛ばさない）。空白や記号は `other` に数える
- `'#'` を読んだら、数えずにループを抜ける

exam08 解答例: rotate.c（配列の回転）

```
    do {
        scanf("%d", &n);
    } while (n < 1);
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
    scanf("%d", &k);
    int shift = k % n;
    for (int i = 0; i < n; i++) {
        printf("%d ", a[(i - shift + n) % n]);
    }
    printf("\n");
```

- n 個回転させると元に戻るので、回す数は `k % n` でよい
- 右に `shift` 個回転させると、i 番目には元の `i - shift` 番目が来る。負にならないように `n` を足してから `% n` を取る
- 配列を実際に動かさず、出力する順番だけを変えている